Android 启动优化
排查指南

版本号: V2.1

发布日期: 2025.09.01

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
V1.0	2021.01.26	AWA0681	初始版本文档
V2.0	2022.11.29	AWA1696	1. 增加常见启动优化排查案例； 2. 增加 Android 13 相关内容； 3. 文档变更为《Android 启动优化排查指南》。
V2.1	2025.09.01	AWA1696	刷新文档说明并修正错误描述；

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1
1.4 相关术语介绍 1

2 Android 启动流程 **3**
2.1 BootLoader 3
2.1.1 IC 上电到 BROM 4
2.1.2 SBOOT 4
2.1.3 UBOOT 4
2.2 Kernel 4
2.3 Android 5

3 测试方法对齐 **6**
3.1 测试条件 6
3.2 启动时间的统计 6
3.2.1 计时方法 6
3.2.1.1 掐表计时 6
3.2.1.2 录像计时 7

4 开机启动排查工具介绍 **8**
4.1 bootchart 8
4.2 bootevent 10
4.3 systrace 11
4.4 logcat 12
4.5 开机启动串口 log 13
4.6 总结 14

5 常见问题排查定位 **15**
5.1 uboot 耗时太长 15
5.2 key_flag 未正确烧写 15
5.3 内核打印等级太高 15
5.4 未关闭 awlog 16
5.5 开机动画过于复杂 16
5.6 分区挂载时间长 17
5.7 SystemServer 部分服务异常 18
5.7.1 PackageManagerService 服务 18
5.7.2 AMS 服务 18

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.7.3 Launcher 太耗时 19
5.8 random 初始化晚导致系统 delay 19
5.8.1 InstallSystemProviders 服务异常 19
5.8.2 boringssl_self_test 服务耗时 20
5.8.3 解决办法 20
5.9 IO 速度慢 21

6 客户端常见优化思路 22
6.1 bootLoader 优化常见措施 22
6.1.1 优化 usb burn key 检测 22
6.1.2 减少 bootloader 打印 22
6.2 kernel 优化常见措施 23
6.2.1 内核模块的裁剪 23
6.2.2 提高 flash 读写性能 23
6.3 Android 优化常见措施 24
6.3.1 CPU 提速 24
6.3.2 IO 提速 24
6.3.3 DDR 提速 25
6.3.4 裁剪 25
6.3.4.1 裁剪 secureboot 25
6.3.4.2 裁剪 GMS 25
6.3.4.3 裁剪 AVB 25
6.3.5 去除不需要的 package 26
6.3.5.1 获取模块名字 26
6.3.5.2 SDK 快速去除 27
6.3.6 使用简单开机动画 27
6.3.7 去除 statusbar 27
6.3.8 优先选择 Go 设备 28
6.3.9 UDISK 分区的文件系统类型 28
6.3.10 使用简单的 launcher 28
6.3.11 减少不必要的预装 apk 28
6.3.12 AVB 优化 28
6.3.13 并行加载驱动 29
6.3.14 延迟加载非必须的模块和服务 29

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　Android 启动流程框架 3
图 4-1　bootchart 实例 9
图 4-2　bootevent 示例 10
图 4-3　systrace 示例 12
图 5-1　bootanim 示例 17
图 5-2　mout 耗时长 1 17
图 5-3　mount 耗时长 2 18
图 6-1　sys_config.fex 22
图 6-2　debug_mode 22
图 6-3　裁剪 AVB 25
图 6-4　package 列表 26
图 6-5　Telecom 模块 26
图 6-6　gomod 使用 27
图 6-7　statusbar 27

广州星翼电子科技有限公司zdyzlinux

1 概述

文档适用于 FAE 或客户如何自查 Android 开机启动时间的问题，包括异常启动、启动优化是否还有空间，启动优化常见思路。

1.1 编写目的

- 本文档提供 Android 快速开机启动优化指南。
- 本文档提供的快速排查指南工程思路如下：启动分解 + 调式工具 + 常见优化措施。
- 本文档仅用于帮助开发人员快速排查和解决初级问题问题，如果按照文档中的指导仍无法解决问题，请联系全志原厂 FAE 进一步协助解决，并提供说明是否按照文档中的排查方案进行过必要的排查，提供排查过程的必要记录和数据。

1.2 适用范围

基于 uboot2018，Android S，A133 安全方案平台编写，适用于 Android P 以后的方案。

1.3 相关人员

本文档主要适用于以下工程师：系统工程师，性能优化工程师，主要面对开机启动优化方面。

1.4 相关术语介绍

术语	解释
OPTEE	是 STMicroelectronics 和 Linaro Security Working Group 开发的安全系统。
Bootloader	启动加载程序。
AMS	ActivityManagerService，Android 核心服务，用于管理 Activity。
PKMS	PackageManagerService，Android 包管理服务。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

术语	解释
AVB	Android Verify Boot，在启动过程进行校验的一种机制。

广州星翼电子科技有限公司zdyzlinux

2 Android 启动流程

Android 完整的启动流程主要包含 3 个阶段，分别为 BootLoader、Kernel 和 Android。

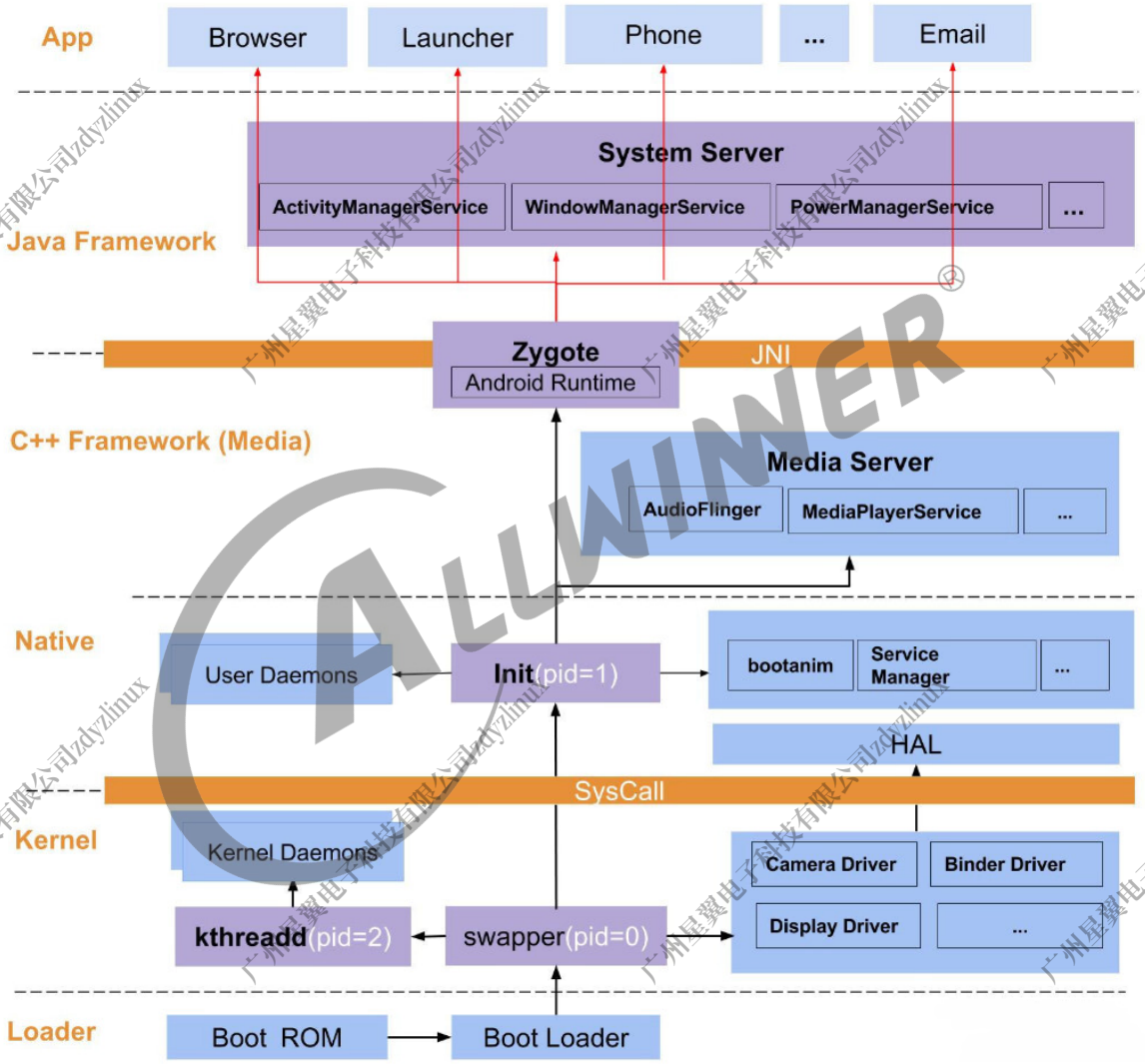

图 2-1: Android 启动流程框架

2.1 BootLoader

BootLoader 阶段主要包含 3 个部分。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.1.1 IC 上电到 BROM

这部分主要包含 IC 上电，一般为长按 Power 键 1s，接着跑 IC 内部固化的 BROM，BROM 的主要任务是检测外部储存是否有 boot 镜像，如有则启动。

2.1.2 SBOOT

1. rsb 通讯以及 pmu 初始化
2. RTC 初始化
3. DRAM 初始化
4. eMMC 初始化
5. 加载并校验 toc1
6. scp 以及 optee 的初始化
   这部分比较耗时的是 toc1 加载校验以及 scp 和 optee 的初始化，大概 400ms。

2.1.3 UBOOT

1. PMU 初始化
2. 多核启动
3. disp 与 eMMC 初始化
4. Secure storage 初始化
5. Usb burn key
6. Verified boot 的加载与校验
7. 启动 kernel

2.2 Kernel

kernel 启动主要有如下 4 类。

1. Initcall 类
2. Platform driver 类
3. Platform device 类
4. Probe 总线类

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.3 Android

android 层的启动主要有如下 6 部分。

1. Init
2. Native、Hal
3. Zygote
4. SystemServer
5. SystemUi
6. Launcher

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 测试方法对齐

对于启动时间，需要判断是否是真的慢了还是只是测试方法不对导致的时间变慢，很多时候，是由于测试手法不对导致了启动变慢。

3.1 测试条件

在启动时间测试时，尽量采用出货固件，如果是 debug 固件则需要对固件进行一系列的相关操作，以达到最优的启动效果，主要如下。

1. 关闭 awlog，通过命令 setprop persist.debug.logpersistd false。
2. loglevel 调整为 4；如果是 Android 13，linux 5.15 则需要调整为 3。
3. 进行烧号，同时要设置写标志位为 1。
4. 第一次和第二次启动完成后，静置 5min 以上，然后通过长按 power 键，正常关机。
5. 如果是 Android 15 及以上平台，则需重启三次，每次静置 5min，并通过长按 power 键正常开关机。

3.2 启动时间的统计

以 A133 平板为例，启动开始时间为按下 power 键上电开始，到锁屏界面出来结束，无锁屏的设备则以 launcher 出来结束。

3.2.1 计时方法

开机时间的统计有掐表计时和录像计时 2 种。

3.2.1.1 掐表计时

掐表计时优点在于测试方便，缺点是精度不高，常用于 debug 或者简单测试阶段。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.1.2 录像计时

录像计时通过手机或高清相机，对启动过程录像，然后通过播放软件（如 potplayer）逐帧分析，找到起始帧和结束帧，相减得到启动时间，录像计时精度高，误差小，通常用于发布阶段。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 开机启动排查工具介绍

4.1 bootchart

bootchart 是一个图形化的开机启动调式工具，它能在系统启动过程自动收集 CPU 占用率、IO 占用率、进程占用等信息，最终以图形化的形式展现出来。

bootchart 需要内核支持 ftrace debug，在 Android P 以后就已支持的很完善，很方便就能够使用，下面是方法。

步骤介绍。

```
adb shell "touch /data/bootchart/enabled"
adb reboot
adb shell "cd data/bootchart;tar -zcf bootchart.tgz *"
adb pull /data/bootchart/bootchart.tgz .

把bootchart.tgz拷贝到服务器
在服务器上执行bootchart bootchart.tgz 生成bootchart.png,注意服务器要装载了bootchart工具
```

下面是一个 bootchart 实例。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

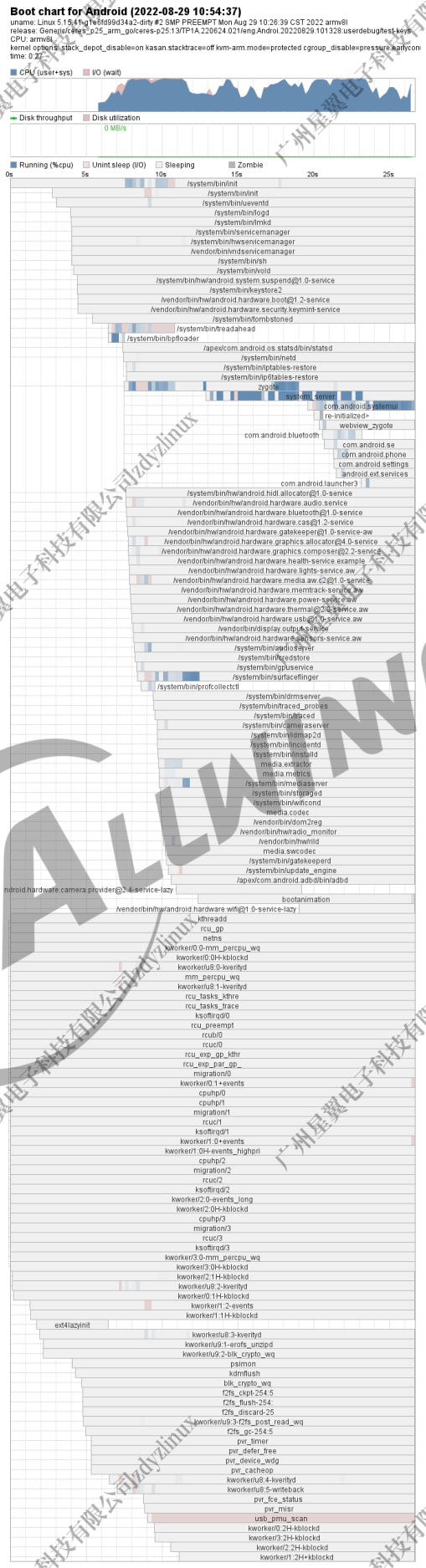

图 4-1: bootchart 实例

版权所有 © 2022 珠海全志科技股份有限公司。保留一切权利

4.2 bootevent

bootevent 是一个 AW 的开机启动调式工具，通过它能够初步判断开机启动有哪些事件，哪些事件耗时多长时间，它是一个开机启动的量化工具，能够帮助工程师分析开机启动过程哪些模块需要优化，但它不能直接实现优化的效果，只能起到排查的作用，具体的优化工作还需要对应模块的工程师去优化。

```
venus-a1:/ # cat /proc/bootevent
----------------------------------------
0          BOOT EVENT (unit:msec)
----------------------------------------
      186.185040 : ON                                  3          4          5
      434.765747 :    1-swapper/0       : initcall: init_menu    20.793000ms
      456.011747 :    1-swapper/0       : initcall: arch_hw_breakpoint_init    15.466083ms
      483.464747 :    1-swapper/0       : probe: probe=platform_drv_probe drv=sun8iw15p1-pinctrl(c0f3fc98)    15.404667ms
1     602.924871 :  2 1-swapper/0       : probe: probe=platform_drv_probe drv=ramoops(c0f37aa8)    24.454500ms
      610.515455 :    1-swapper/0       : initcall: of_platform_default_populate_init    153.933374ms
      630.943871 :    1-swapper/0       : initcall: param_sysfs_init    18.970916ms
      646.651538 :    1-swapper/0       : initcall: pm_sysrq_init    15.629500ms
      692.190496 :    1-swapper/0       : initcall: usb_init    18.306000ms
      744.694371 :    1-swapper/0       : probe: probe=platform_drv_probe drv=pmu1736-regulator(c101fa04)    30.764041ms
      744.794037 :    1-swapper/0       : initcall: pmu1736_regulator_initcall    31.132375ms
      793.230371 :    1-swapper/0       : probe: probe=platform_drv_probe drv=ion-sunxi(c10320f4)    40.734667ms
      793.347787 :    1-swapper/0       : initcall: sunxi_ion_init    41.149750ms
      833.801495 :    1-swapper/0       : initcall: bt_init    32.528208ms
      864.405745 :    1-swapper/0       : initcall: tracer_init_tracefs    22.171500ms
      912.793453 :    1-swapper/0       : initcall: event_trace_init    48.338541ms
```

图 4-2: bootevent 示例

1. 第一列表示启动时间，单位为毫秒。
2. 第二列表示进程号以及进程名称。
3. 第三列表示哪种类型的启动时间，如 initcall、probe、INIT、Zygote、PMS、AMS、System-Server、BOOT_Animation 等。
4. 第四列表示具体函数。
5. 第五列表示函数耗时，只记录大于 15ms 的函数。

注意：只有内核层的函数有耗时时间戳，安卓层需自行计算时间戳。

案例参考。

```
ceres-b3:/ # cat /proc/bootevent
----------------------------------------
0 BOOTEVENT (unit:msec)
----------------------------------------
      2.590374 : ON
     50.498874 :   1-swapper/0     : initcall: of_platform_default_populate_init   26.289917ms
    114.685832 :   1-swapper/0     : initcall: tracer_init_tracefs   51.512125ms
    277.561790 :   1-swapper/0     : initcall: populate_rootfs   160.092416ms
    326.208123 :   1-swapper/0     : initcall: jent_mod_init   37.240125ms
    391.569331 :   1-swapper/0     : Kernel_init_done
    637.936747 :  32-kworker/0:1   : probe: probe=platform_drv_probe drv=axp2101-regulator((ptrval))   68.178583ms
    675.530997 :  32-kworker/0:1   : probe: probe=platform_drv_probe drv=axp803-battery-power-supply((ptrval))
      37.064292ms
   2439.267198 : 169-init         : INIT:early-init
   2652.876531 : 169-init         : INIT:late-init
   2676.109072 : 169-init         : INIT:Mount_START
   2720.540197 : 169-init         : INIT:Mount_END
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
 2855.573697 : 169-init        : INIT:post-fs
 2985.410863 : 169-init        : INIT:late-fs
 4237.405025 : 169-init        : INIT:post-fs-data
 5598.351394 : 169-init        : INIT:boot
 8329.327176 : 299-surfaceflinger : BOOT_Animation:START
 8436.521675 : 265-main        : Zygote:ZygotePreload Start
 9101.595381 : 265-main        : Zygote:ZygotePreload took 669ms.
//·····
 9753.370754 : 459-system_server  : Android:SysServerInit_START
10134.289586 : 459-system_server  : Android:loadPreinstalledSystemFontMap start
10135.411086 : 459-system_server  : Android:loadPreinstalledSystemFontMap end
10136.205002 : 459-system_server  : Android:InitBeforeStartServices end
10884.714624 : 477-Thread-2       : Zygote:preload 7337 classes in 1132ms.
10906.465874 : 476-Thread-4       : Zygote:ThreadPreadloadClass took 1156ms.
11072.924540 : 459-system_server  : Android:PackageManagerService_Start
11528.392497 : 459-system_server  : Android:PMS_scan_START
//·····
12188.781911 : 459-system_server  : Android:PMS_scan_END
12273.396911 : 459-system_server  : PMS:reconcileAppsDataLI start
12401.181410 : 459-system_server  : PMS:reconcileAppsDataLI end
12485.043577 : 459-system_server  : Android:PMS_READY
14104.582112 : 265-main        : Zygote:preload 7337 classes in 1097ms.
14242.994028 : 265-main        : Zygote:preload 64 obtatin resources in 113ms.
14272.970861 : 265-main        : Zygote:preload 41 resources in 30ms.
14922.524151 : 459-system_server  : AMS:systemReady
14932.834776 : 459-system_server  : AMS:AMS_READY
15392.440190 : 459-system_server  : SystemServer:NetworkStatsService systemReady
15537.171565 : 459-system_server  : SystemServer:ConnectivityService systemReady
16061.220313 : 459-system_server  : SystemServer:PhaseThirdPartyAppsCanStart
16674.718686 : 492-ActivityManager : AM_PROC_START:starProcess {com.android.settings/com.android.settings.
    FallbackHome}
16684.018936 : 459-system_server  : Android:SysServerInit_END
17502.144057 : 640-ndroid.systemui : SystemUIService:Starting SystemUI services
//·····
20417.465255 : 340-Binder:299_1   : BOOT_Animation:END
20604.883879 : 484-android.display : AMS:set sys.boot_completed true
20648.169129 : 484-android.display : ActivityManager:Finished locked boot
22927.796078 : 915-droid.launcher3 : Launcher:onCreate start
24614.942072 : 915-droid.launcher3 : Launcher:onCreate end
24634.233780 : 915-droid.launcher3 : Launcher:onResume start
24635.814655 : 915-droid.launcher3 : Launcher:onResume end
24636.144572 : OFF
```

说明

GKI 平台由于 Google 的要求，无法修改内核，故无法统计驱动模块的加载耗时。

4.3 systrace

systrace 是 Google 支持的一款工具，用于系统启动过程中收集 kernel 以及 Android 的 trace 信息，用于分析开机启动，以下方法适用于 AndroidR 之后的版本。

第一步，打开 trace 属性，然后重启进入 uboot 命令行。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
console:/ # setprop  persist.debug.traced.enable 1
console:/ # getprop persist.debug.traced.enable
1
console:/ # reboot uboot
```

第二步，设置 env 环境变量。

```
=> printenv
trace_buf_size=64M
trace_enable=0
trace_event=sched_wakeup,sched_switch,sched_blocked_reason,sched_cpu_hotplug,block,ext4
=> setenv trace_enable 1
=> saveenv
Saving Environment to SUNXI_FLASH... Writing to env...
OK
=> printenv
trace_buf_size=64M
trace_enable=1
trace_event=sched_wakeup,sched_switch,sched_blocked_reason,sched_cpu_hotplug,block,ext4
=> reset
```

第三步，抓 boottrace 数据。

```
adb root && adb shell atrace --async_stop -z -c -o /data/local/tmp/boot_trace
adb pull /data/local/tmp/boot_trace .

//然把boot_trace放到anroid/external/chromium-trace/目录下
$cd  anroid/external/chromium-trace/
$./systrace.py --from-file trace/boot_trace -o a133_r_systrace_v1.html
```

将生成的 boot_trace.html 拿出来，使用 chrome 浏览器打开即可查看到完整的启动 systrace 了。

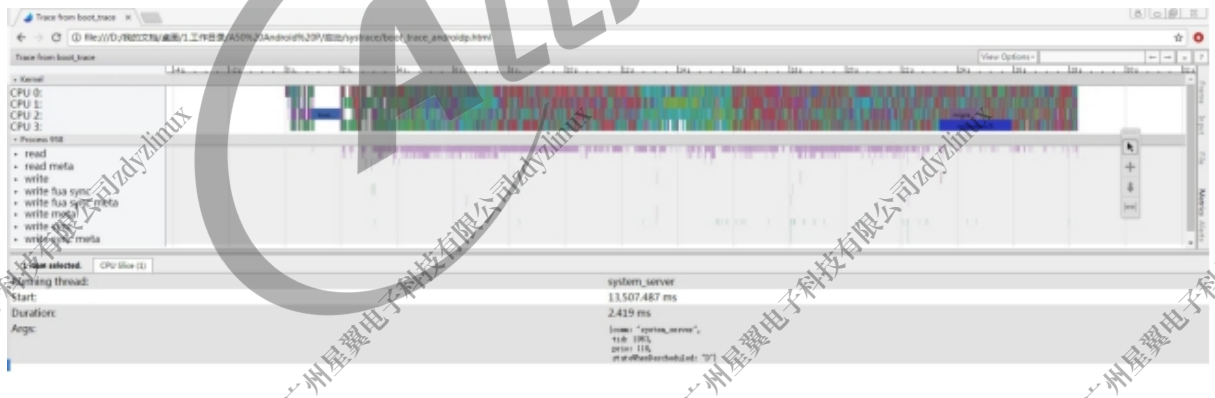

图 4-3: systrace 示例

说明

不同 Android 版本里面的 chromium-trace 可能存在差异，请使用对应 SDK 仓库下的 chromium-trace 进行 trace 的转换。

4.4 logcat

即使有了前面的工具，我们还是离不开 logcat 最原始的分析调式工具，一般而言，通过 logcat > /data/logcat.txt，将整个启动 logcat 抓出来。还可以通过 logcat -b events | grep boot 来抓取重要的 events。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

logcat 分析启动的关键字。

1. 搜索”phase”，来判断启动是否完成，如果到了 1000 表示开机已经结束，如下。

```
console:/ # logcat | grep phase
SystemServiceManager: Starting phase 100
SystemServiceManager: Starting phase 480
SystemServiceManager: Starting phase 500
SystemServiceManager: Starting phase 520
SystemServiceManager: Starting phase 550
SystemServiceManager: Starting phase 600
SystemServiceManager: Starting phase 1000
```

2. 搜索”SystemServerTiming”，分析 system server 耗时，如下。

```
console:/ # logcat | grep SystemServerTiming
SystemServerTiming: InitBeforeStartServices took to complete: 606ms
SystemServerTiming: ReadingSystemConfig took to complete: 8ms
SystemServerTiming: StartInstaller took to complete: 14ms
SystemServerTiming: DeviceIdentifiersPolicyService took to complete: 2ms
SystemServerTiming: StartActivityManager took to complete: 539ms
```

3. 搜索”SystemUIBootTiming”，分析 systemui 耗时，如下。

```
SystemUIBootTiming: DependencyInjection took to complete: 1ms
SystemUIBootTiming: StartServices com.android.keyguard.KeyguardBiometricLockoutLogger took to complete: 4ms
SystemUIBootTiming: StartServices com.android.systemui.LatencyTester took to complete: 25ms
SystemUIBootTiming: StartServices com.android.systemui.statusbar.notification.interruption.
        KeyguardNotificationVisibilityProvider took to complete: 94ms
SystemUIBootTiming: StartServices com.android.systemui.statusbar.phone.CentralSurfaces took to complete: 1377ms
SystemUIBootTiming: StartServices took to complete: 1836ms
```

4. 搜索”StartServices took to complete”，来判断 system server 以及 systemui 整个启动加载总耗时，如下。

```
console:/ # logcat | grep "StartServices took to complete"
SystemServerTiming: InitBeforeStartServices took to complete: 606ms
SystemServerTiming: StartServices took to complete: 6816ms
SystemUIBootTiming: StartServices took to complete: 3893ms
```

4.5 开机启动串口 log

因为开机启动除了涉及 kernel、Android 之外，还涉及 uboot，所以还需要连接串口抓取开机启动的 log，便于全面分析开机启动时间。

抓取方法：机器处于关机状态，连接串口、断开 USB，按下 power 键，系统开机，抓取从开机到结束的完整串口 log。

如果在 loglevel 为 3 或者 4 的情况下，串口日志较少，则可以在开机完成后，使用 dmesg > /data/dmesg 将所有串口日志打印到文件中，再通过 adb pull 把对应的 log 抓出来分析。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.6 总结

以上五个步骤均为 Android 开机启动排查的必要步骤，分析开机启动时间，请务必按照上述五个重要步骤抓取相关数据，综上包含。

- bootchart
- bootevent
- systrace
- logcat
- 串口启动 log

说明

正常情况下不需要提供 systrace，必要情况需要提供，其他四个必须提供。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 常见问题排查定位

5.1 uboot 耗时太长

```
[01.411]usb burn from boot
delay time 0
weak:otg_phy_config
[01.421]usb prepare ok
[01.637]usb sof ok
[01.638]usb probe ok
[01.640]usb setup ok
set address 0x39
set address 0x39 ok
try to update
[04.647]do_burn_from_boot usb : have no handshake
[04.651]boot_gui_init:start
[04.656]set disp.dev2_output_type fail. using defval=0
[04.795]boot_gui_init:finish
[04.798]bmp_name=bootlogo.bmp
......
[05.456]Starting kernel ...
```

1. 通过 usb probe ok 可得知插着 USB 测试。
2. 通过 have no handshake 可知没有烧号，有一个等待烧号的过程。

结论：上述表述 uboot 总时间为 5.456s，其中 1.64s~4.647s 是空转时间，原因是 key_flag 没烧，且插着 usb 测试。

5.2 key_flag 未正确烧写

测试开机启动性能的机器应当按照模拟量产出货的场景，量产出货时应当烧写 SN，烧写 SN 的时候应当烧写 key_flag，具体方法参考《DragonSN 使用说明文档》。

5.3 内核打印等级太高

正常出货量产的样机都应该是 user 固件，user 固件默认打印等级是 4，若测试用的是 userdebug 固件，应当改为 4，修改方法如下。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
android/device/softwinner/ceres$
 diff --git a/common/system/env.cfg b/common/system/env.cfg
 index 31d1148..1560541 100644
 --- a/common/system/env.cfg
 +++ b/common/system/env.cfg
 @@ -7,7 +7,7 @@ console=ttyAS0,115200
  nand_root=/dev/nand0p4
  mmc_root=/dev/mmcblk0p4
  init=/init
 -loglevel=8
 +loglevel=4
  cma=8M
  mac=
  wifi_mac=
```

说明

如果是 Android 13 linux-5.15 内核则应改为 3，原因是当前的内核存在 BUG，会出现异常 BPF 打印影响启动时间。

5.4 未关闭 awlog

若测试采用的是 userdebug 固件而不是 user 固件，则需要手动关闭 awlog 后台功能（user 固件无需此步骤），修改方法如下。

1. 编译时关闭 awlog。

```
android/device/softwinner/ceres$
--- a/common/system/config.mk
+++ b/common/system/config.mk
@@ -43,7 +43,7 @@ PRODUCT_SYSTEM_DEFAULT_PROPERTIES += \
    persist.display.enhance_mode=0 \

 ifneq (,$(filter userdebug eng,$(TARGET_BUILD_VARIANT)))
-PRODUCT_DEBUG := true
+#PRODUCT_DEBUG := true
 PRODUCT_SYSTEM_DEFAULT_PROPERTIES += \
    persist.sys.usb.config=adb \
    ro.adb.secure=0 \
```

2. 运行时关闭 awlog。

```
setprop persist.debug.logpersistd false
reboot
```

5.5 开机动画过于复杂

在某客户反馈的启动时间 2G 和 4G 的实际差异很大的问题中，排查发现 2G 样机的动画太过复杂，与 4G 方案的并不一致，后面客户再发过来的 2G 固件的动画又是正常的了。影响有 6s 左右。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

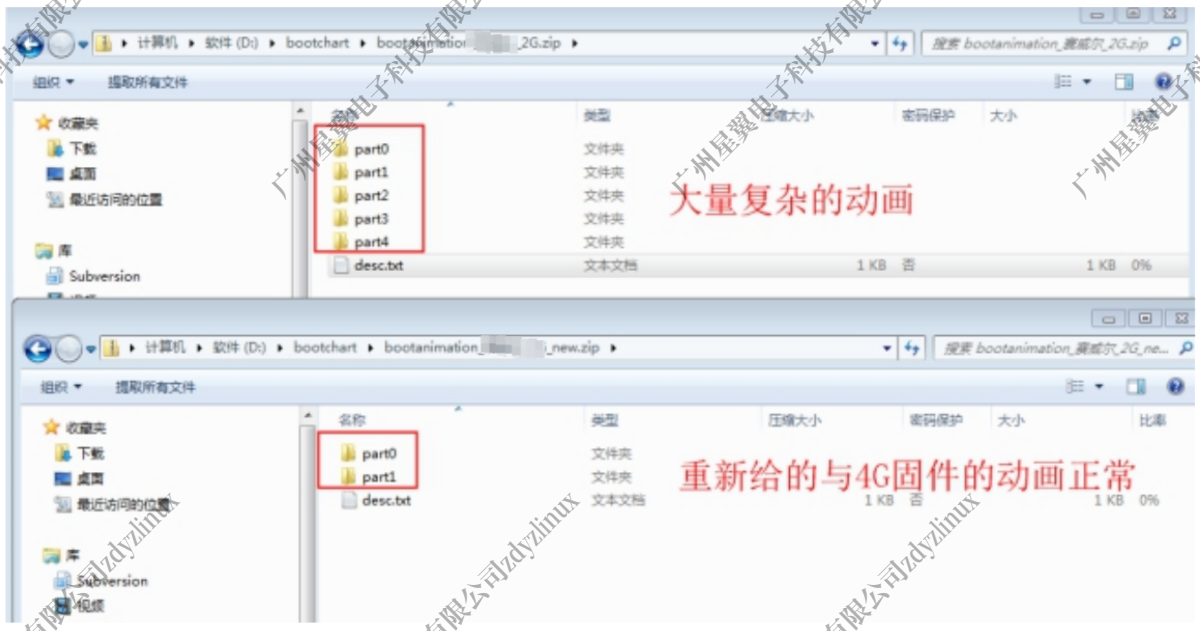

图 5-1: bootanim 示例

解决方法：使用简单的动画，不要使用太过复杂的动画，客户重新给的固件已经不是之前的复杂动画，所以不用我们解决。

5.6 分区挂载时间长

正常的挂载时间 bootevent 表现。

```
2397.512073 : 162-init        : INIT:Mount_START
2654.166030 : 162-init        : INIT:Mount_END
```

异常的挂载时间 bootevent 表现。

```
5934.974560 : 1255-init        : INIT:Mount_START
10324.307251 : 1255-init       : INIT:Mount_END
```

图 5-2: mout 耗时长 1

此时应该查看 logcat 打印。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
05-04 02:10:15.997  1887  1887 I vold    : Vold 3.0 (the awakening) firing up
05-04 02:10:15.997  1887  1887 E cutils-trace: Error opening trace file: No such file or directory (2)
05-04 02:10:15.998  1887  1887 D vold    : Detected support for: ext4 f2fs vfat
05-04 02:10:16.006  1887  1887 D vold    : Found unmanaged dm device named product
05-04 02:10:16.006  1887  1887 D vold    : Found unmanaged dm device named vendor
05-04 02:10:16.006  1887  1887 D vold    : Found unmanaged dm device named system
05-04 02:10:16.007  1887  1887 W vold    : Failed to LOOP_GET_STATUS64 /dev/block/loop5: No such device or address
05-04 02:10:16.007  1887  1887 W vold    : Failed to LOOP_GET_STATUS64 /dev/block/loop7: No such device or address
05-04 02:10:16.007  1887  1887 W vold    : Failed to LOOP_GET_STATUS64 /dev/block/loop0: No such device or address
05-04 02:10:16.007  1887  1887 W vold    : Failed to LOOP_GET_STATUS64 /dev/block/loop2: No such device or address
05-04 02:10:16.007  1887  1887 W vold    : Failed to LOOP_GET_STATUS64 /dev/block/loop4: No such device or address
05-04 02:10:16.008  1887  1887 W vold    : Failed to LOOP_GET_STATUS64 /dev/block/loop3: No such device or address
05-04 02:10:16.008  1887  1887 W vold    : Failed to LOOP_GET_STATUS64 /dev/block/loop6: No such device or address
05-04 02:10:16.008  1887  1887 W vold    : Failed to LOOP_GET_STATUS64 /dev/block/loop1: No such device or address
05-04 02:10:16.014  1887  1887 D vold    : VoldNativeService::start() completed OK
05-04 02:10:20.141  1884  1884 I hwservicemanager: hwservicemanager is ready now.
05-04 02:10:20.212  1887  1898 I vold    : Event queue push back action:1, dev:/devices/virtual/block/ram11
05-04 02:10:20.213  1887  1896 I vold    : Event queue pop front action:1, dev:/devices/virtual/block/ram11
05-04 02:10:20.213  1887  1898 I vold    : Event queue push back action:1, dev:/devices/virtual/block/dm-1
05-04 02:10:20.213  1887  1896 I vold    : Event queue pop front action:1, dev:/devices/virtual/block/dm-1
05-04 02:10:20.216  1887  1898 I vold    : Event queue push back action:1, dev:/devices/virtual/block/ram2
05-04 02:10:20.216  1887  1896 I vold    : Event queue pop front action:1, dev:/devices/virtual/block/ram2
05-04 02:10:20.217  1887  1898 I vold    : Event queue push back action:1, dev:/devices/virtual/block/loop1
05-04 02:10:20.217  1887  1898 I vold    : Event queue push back action:1, dev:/devices/virtual/block/ram0
```

分区挂载耗时异常

图 5-3: mount 耗时长 2

5.7 SystemServer 部分服务异常

5.7.1 PackageManagerService 服务

SystemServer 启动服务的时间耗时很长。

```
SystemServerTiming: InitBeforeStartServices took to complete: 355ms
SystemServerTiming: StartServices took to complete: 10392ms
SystemUIBootTiming: StartServices took to complete: 1198ms
```

分析发现是 PKMS 的耗时太长了。

正常的 PKMS 的启动时间是 1.6s 以内。

```
SystemServerTiming: StartPackageManagerService took to complete: 1600ms
```

异常的 PKMS 时间。

```
SystemServerTiming: StartPackageManagerService took to complete: 4330ms
```

引起这部分异常的主要原因大多可能是因为预装的应用太多，Overlay apk 太多导致。

5.7.2 AMS 服务

正常的 AMS 时间是 500~900ms 左右。

```
SystemServerTiming: ActivityManagerStartApps took to complete: 560ms
```

异常的 AMS 时间。

```
SystemServerTiming: ActivityManagerStartApps took to complete: 2258ms
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

注意：AMS 启动时间与 Android 版本息息相关，版本越低，时间越短，版本越高，随时 OS 复杂度增加，时间越长。

5.7.3 Launcher 太耗时

原生的 Launcher 选用的是 Launcher3，但很多产品都会选择自定义 Launcher，如 TVLauncher 等。

正常的 Launcher 耗时大概是在 2s 以内。

```
ActivityTaskManager: Displayed com.swe.dgbluancher/.MainActivity: +1s864ms
```

异常的 Launcher 耗时。

```
ActivityTaskManager: Displayed com.droidlogic.mboxlauncher/.Launcher: +4s776ms
```

Launcher 的耗时，可以通过 bootevent 排查。

```
23009.342536 :  980-droid.launcher3 : Launcher:onCreate start
24545.896156 :  980-droid.launcher3 : Launcher:onCreate end
```

5.8 random 初始化晚导致系统 delay

random 是内核的一个随机数发生器，主要用于生成随机数，在 Android 启动过程会有 random 的初始化，random 初始化主要依赖于系统熵池的熵信息数量，如果 random 的初始化时间太晚，Android 层很多依赖于 random 的服务就会发生 delay，从而导致启动时间变长，其中主要注意的是 random 初始化完成的打印。

```
random: crng init done
```

5.8.1 InstallSystemProviders 服务异常

InstallSystemProviders 服务的初始化过程需要获取 UUID，而 UUID 需要依赖于 random 的初始化完成实现。

正常的InstallSystemProviders服务启动。

```
SystemServerTiming: InstallSystemProviders
SystemServerTiming: InstallSystemProviders took to complete: 265ms
```

而异常InstallSystemProviders的耗时如下。

```
SystemServer: InstallSystemProviders
SystemServerTiming: InstallSystemProviders took to complete: 4921ms
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.8.2 boringssl_self_test 服务耗时

在 Android 13 中，通过 bootevent 发现 early-init 的完成时间比较晚，差不多延迟了有 2s，

异常打印如下。

```
console:/ # cat proc/bootevent
----------------------------------------
0 BOOTEVENT (unit:msec)
----------------------------------------
      4.960166 : ON
    388.179331 :   1-swapper/0     : Kernel_init_done
   5478.565020 :  179-init         : INIT:early-init
```

通常来说，early-init 的时间一般是在 3s 左右的，太长就说明有问题了，排查定位为 boringssl_self_test 服务异常。

```
[   9.290770][    T1] init: SVC_EXEC service 'boringssl_self_test32' pid 196 (uid 0 gid 0+0 context default) started; waiting...
[  11.792049][    C2] random: crng init done
[  11.854080][    T1] init: Service 'boringssl_self_test32' (pid 196) exited with status 0 waiting took 2.569000 seconds
```

多次实验发现，每次都会卡在 random 初始化完成后，大胆怀疑 boringssl_self_test32 服务是在等待 random 完成，最终追溯源码也确实发现是在等待 random。

5.8.3 解决办法

对于 random 初始化慢的问题，主要有以下方法。

1. 增加一个硬件随机数发生器，可有效解决该问题，但会带来成本提升。
2. 对于非 GKI 设备，可直接修改内核，采用 urandom 替代 random，修改如下。

```
diff --git a/drivers/char/random.c b/drivers/char/random.c
index ebe86de9d0ac..207713fb9c8a 100644
--- a/drivers/char/random.c
+++ b/drivers/char/random.c
@@ -462,7 +462,7 @@ static struct crng_state primary_crng = {
 */
 static int crng_init = 0;
 static bool crng_need_final_init = false;
-#define crng_ready() (likely(crng_init > 1))
+#define crng_ready() (likely(crng_init > 0))
 static int crng_init_cnt = 0;
 static unsigned long crng_global_init_time = 0;
 #define CRNG_INIT_CNT_THRESH (2*CHACHA_KEY_SIZE)
```

3. 针对 GKI 设备，主要解决方法有 2 个。

- 在 vendorhook 驱动初始化过程中增加中断数，快速增加 random 熵池，加速初始化，当前中断数是 12500。
- 在上述方法无法使用，或使用了不佳的情况，则需要针对源码进行修改，尝试优化。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.9 IO 速度慢

若选用 nand flash，因 nand flash 本身的 IO 随机读性能不如 eMMC 的，IO 速度会导致启动时间变长，一般的 nand flash 随机读性能在 2500iops 左右，eMMC flash 的随机读性能在 3500iops 以上。

说明

可以使用 AndroidBench 应用进行 IO 速度测试。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 客户端常见优化思路

6.1 bootLoader 优化常见措施

6.1.1 优化 usb burn key 检测

默认 SDK 需要打开 burn_key，导致每次开机都需要检查 usb 是否存在，假设 usb 不存在，当前整个 usb 检查过程耗时 800ms，原因是检测 usb 的方法实现有问题，不能真正判断 usb 是否存在，而导致了延时的存在。修复检测 usb 的方法后，可以优化开机启动时间 800ms。

也可以将 sys_config.fex 的 burn_key 设置为 0。

```
;-----------------------------
[target]
boot_clock        = 1008
storage_type      = -1
burn_key          = 1
dragonboard_test  = 0
```

图 6-1: sys_config.fex

6.1.2 减少 bootloader 打印

将 sys_config.fex 的 debug_mode 改为 0 后，整个 uboot 的打印基本就没有了，主要是减少了 boot0 的打印，因为那会频率低，串口输出慢，开机时间节省大概 130ms。Debug_mode=1 是正常的发布打印等级，如果要看到更多的 uboot log，需要设置为 4，如果设置为 0，则开机启动时间能减少 130ms。

```
;--------------------
[platform]
eraseflag    = 1
next_work    = 3
debug_mode   = 1
```

图 6-2: debug_mode

目前这个修改没有集成，需要的话客户自己选择。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.2 kernel 优化常见措施

6.2.1 内核模块的裁剪

内核模块的裁剪分两部分，第一部分是 debug 模块，debug 主要是针对一些开机启动无关的 debug，例如 slub_debug，这里要说的是 tracers 的 debug，对于一些不需要过 GMS 的平台，针对发布的 SDK 把 tracers 关了能节省大概 500ms 的时间。由于 A50 的 CTS 测试要求要打开 tracers，这里保持不修改，但是对于车载产品是可以删除的。

```
Kernel hacking --->
  [*] Tracers ---> //把*去掉表示关掉tracers
```

第二部分指的是一些不必要的驱动模块，尤其是耗时的模块，可以通过 make menuconfig 的方法把不必要的硬件驱动去除，例如 A50-a3 有部分 sensor 是没有使用的，那么可以将未使用的模块不要编译进内核。

6.2.2 提高 flash 读写性能

A50 有 nand 和 eMMC 两种 flash 介质，其中 eMMC 还有高速模式（HS400-100MHz），通过实验发现，nand flash 介质对启动影响特别大，由于 nand 的连续读速度慢（ext4:20MB/s，squashfs：16MB/s），开机启动时间长达 60s 左右。而 eMMC 即使在 SDR50 的模式下连续读（65MB/s），开机启动时间 30s 左右。当把 eMMC 改为高速模式后，开机启动时间有大概 2.2s 的优化，主要原因是整个启动过程读 system vendor 分区的速度快了，例如 zygote 加载 classes，pms scan 速度加快等。

修改 eMMC 为高速的模式的补丁为：

```
--- a/pack/chips/{chip}/configs/{board}/sys_config.fex
+++ b/pack/chips/{chip}/configs/{board}/sys_config.fex
@@ -58,8 +58,8 @@ aldo5_vol       = 2500
 ;------------------------------------------------------------------------------
 ;gpio_bias pc_bias for emmc high speed mode use
 ;------------------------------------------------------------------------------
-;[gpio_bias]
-;pc_bias       = "axp858:bldo5:1800"
+[gpio_bias]
+pc_bias       = "axp858:bldo5:1800"

 [card_boot]
 logical_start   = 40960
@@ -100,7 +100,7 @@ sdc_d7        = port:PC16<3><1><3><default>
 sdc_emmc_rst    = port:PC01<3><1><3><default>
 sdc_ds        = port:PC00<3><1><3><default>
 sdc_ex_dly_used = 2
-sdc_io_1v8 = 0
+sdc_io_1v8 = 1
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.3 Android 优化常见措施

6.3.1 CPU 提速

默认发布的 SDK，开机启动过程均是使用 performance 调频策略，即以 IC 支持的最高频率运行，如 1.5G，排查方法，可在开机过程，连接串口，输入 cpu_monitor -c 1000 命令查看是否以最高频率运行。

将 cpu 默认策略修改为 performance 策略，有 2 种修改方式。

- 在 Android 13 linux-5.15 内核，如果是 GKI 固件，由于 google 的 GKI 镜像默认使用的是 schedutil 策略，无法直接修改内核使用 performance 策略，因此可以通过 module_param 传参的方式修改，改动如下。

```
diff --git a/common/system/env.cfg b/common/system/env.cfg
index 6c968b8..9e9fa16 100644
--- a/common/system/env.cfg
+++ b/common/system/env.cfg
@@ -14,6 +14,7 @@ wifi_mac=
 bt_mac=
 specialstr=
 rcu_idle_gp_delay=1
+default_governor=performance
 force_normal_boot=1
 slot_suffix=_a
 keybox_list=widevine,ec_key,ec_cert1,ec_cert2,ec_cert3,rsa_key,rsa_cert1,rsa_cert2,rsa_cert3
@@ -22,8 +23,8 @@ trace_buf_size=64M
 trace_event=sched_wakeup,sched_switch,sched_blocked_reason,sched_cpu_hotplug,block,ext4
 trace_enable=0
 #set kernel cmdline if boot.img or recovery.img has no cmdline we will use this
-setargs_nand=setenv bootargs rcutree.rcu_idle_gp_delay=${rcu_idle_gp_delay}
+setargs_nand=setenv bootargs rcutree.rcu_idle_gp_delay=${rcu_idle_gp_delay} cpufreq.default_governor=${
    default_governor}
 #nand command syntax: sunxi_flash read address partition_name read_bytes
 #0x4007f800 = 0x40080000(kernel entry) - 0x800(boot.img header 2k)
 boot_normal=sunxi_flash read 50000000 boot;bootm 50000000
```

- 对于非 GKI 固件，可直接修改 defconfig，将默认策略改为 performance 即可。

6.3.2 IO 提速

eMMC 比 Nand 的 IO 速度要好，首选 eMMC，一般要求随机读性能要达到 4000iops 以上较为正常，容量对随机读有正向影响，容量越大，随机读性能越好。

eMMC 的频率模式自查。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
dmesg命令：
[ 1.467959] sunxi-mmc 4022000.sdmmc: sdc set ios:clk 100000000Hz bm PP pm ON vdd 23 width 8 timing MMC-HS400
    dt B
```

一般 eMMC 都应该跑 HS400 100MHz。

说明

可以使用 AndroidBench 应用进行 IO 速度测试。

6.3.3 DDR 提速

不同的方案，DDR 的物料不一样，有些差的物料只能跑 576MHz，正规 LPDDR4 的物料能跑 792MHz，一般来说 DDR 频率，每增加 100MHz，整体的开机启动时间约减少 1.2s，即 576 与 792 的启动时间约相差 2.4s 左右。

6.3.4 裁剪

裁剪优化思路适用于非 GMS 平台系统，若有 GMS 要求，跳过此章节。

6.3.4.1 裁剪 secureboot

非 GMS 平台，无 secureboot 要求，属于 normal boot，打包时使用 pack -d，不带-v 参数即可。secureboot 因需要对证书进行加载校验、以及 AVB 功能的验证等，相比 normal boot 要多出 1s 左右。

6.3.4.2 裁剪 GMS

非 GMS 平台，方案编译去除 GMS 包。

6.3.4.3 裁剪 AVB

非 GMS 平台，且对 System 分区无安全要求的，可去掉此功能，但对 System 分区有防篡改要求的不能去掉。

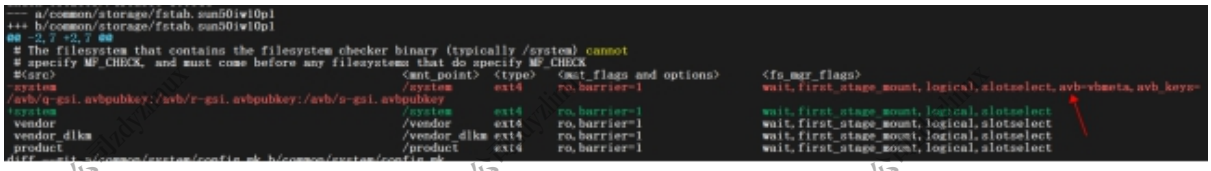

图 6-3: 裁剪 AVB

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.3.5 去除不需要的 package

在 android 的版本升级过程中，谷歌会加入很多的 package，而这些 package 一方面会减慢开机速度，另一方面会占用许多内存，因此在部分平台，可以对 package 进行裁剪。

6.3.5.1 获取模块名字

可以在 adb 通过如下命令，查看当前机器里面的 package。

```
pm list package
```

参考如下。

```
ceres-p25:/ # pm list package
package:com.google.android.providers.media.module
package:com.google.android.overlay.gmsconfig.go
package:com.google.android.overlay.modules.permissioncontroller.forframework
package:com.google.android.apps.searchlite
package:com.google.android.apps.mediahome.launcher
package:com.google.android.safetycenter.resources
package:com.google.android.apps.youtube.music
package:com.softwinner.camera
package:com.android.internal.display.cutout.emulation.hole
package:com.android.settings
package:com.android.bips
package:com.google.android.partnersetup
package:com.android.internal.systemui.navbar.gestural_narrow_back
package:com.softwinner.theme.color
```

图 6-4: package 列表

可根据上述包名，通过下面命令获取获取 SDK 里面对应的模块名字。

```
dumpsys package xxx | grep resourcePath

//示例
dumpsys package com.android.server.telecom | grep resourcePath
```

参考如下，其中 Telecom 就是对应的模块名字。

```
ceres-p25:/ # dumpsys package com.android.server.telecom | grep resourcePath
    resourcePath=/system/priv-app/Telecom
ceres-p25:/ #
```

图 6-5: Telecom 模块

在 sdk 中，执行过 source 和 lunch 选择编译版型后，就可以通过 gomod xxx(模块名) 来直接跳转相应的路径了。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
android/device/softwinner/ceres$
android/device/softwinner/ceres$
android/device/softwinner/ceres$ gomod Telecom
android/packages/services/Telecomm$
android/packages/services/Telecomm$
android/packages/services/Telecomm$
```

图 6-6: gomod 使用

说明

部分早期的 Android 版本可能不支持 gomod 命令，具体以 SDK 里执行情况为准。

6.3.5.2 SDK 快速去除

通常而言，去除非必要的 package，往往是需要到指定的模块里面，把编译的 mk 或者 bp 移除，这样的缺点就是改动大，后期还原困难，aw 针对这种情况，设计了GLOBAL_REMOVED_PACKAGES宏，用于简单快速的去除不需要的 package，而无需修改源码。

使用该宏，只需要在方案配置里，将不需要的模块名字，加到该宏后面即可，参考范例如下。

```
GLOBAL_REMOVED_PACKAGES += Bluetooth
```

如上，就是在方案的 mk 里面去除了蓝牙模块，而不需要对源码做任何改动。

6.3.6 使用简单开机动画

若系统无开机动画要求，可去掉开机动画，如音箱产品无开机动画要求，或者使用最简单的开机动动画。

方法：方案目录下搜索 bootanimation.zip，在.mk 文件删除即可。

6.3.7 去除 statusbar

隐藏 StatusBar 可以节省 2s 的启动时间，具体修改如下。

```
--- a/overlay/frameworks/base/packages/SystemUI/res/values/config.xml
+++ b/overlay/frameworks/base/packages/SystemUI/res/values/config.xml
@@ -22,4 +22,11 @@
     <!-- disable the automatic dimming animation to save cpu cycles-->
     <bool name="config_navigation_bar_enable_auto_dim_no_visible_wallpaper">false</bool>

+    <!-- Empty stub status/navigation bar -->
+    <!-- com.android.systemui.statusbar.tv.TvStatusBar or com.android.systemui.statusbar.phone.StatusBar -->
+    <string name="config_statusBarComponent" translatable="false">com.android.systemui.statusbar.tv.TvStatusBar</string>
+
```

图 6-7: statusbar

版权所有 © 珠海全志科技股份有限公司。保留一切权利

如果产品没有必要有 StatusBars，可以去掉 StatusBar，但对于平板类是必须的。

6.3.8 优先选择 Go 设备

系统方案能选择 Go 设备最好选择 Go 设备，因为非 Go 设备新增太多要求，例如 apex、mainline 等功能限制，建议优先选择 Go 设备，影响时间大约在 3~4s 左右。

mainline，即 Google 针对部分仓库进行限制修改，例如 art 仓库，不允许厂家自行修改 art 仓库下的代码，若要修改必须上传到 Google，Google 合并才有效果，否则修改无效。

6.3.9 UDISK 分区的文件系统类型

部分产品没有电池，关机常常不是按 power 键关机，换句话说不是走正常的 reboot/poweroff 流程，而是直接掉电关机，这样会导致系统再次起来后，会在 mount 阶段多一个 check 流程，相当于修复文件系统，通过实验发现 Android P 引进的 f2fs 文件系统在 check 阶段耗时 2s，相比之下 ext4 在这方面比较好，只需要 0.7s。所以，针对无电池的产品，建议 UDISK 使用 ext4 文件系统。

6.3.10 使用简单的 launcher

原生 launcher3 比较复杂，如果针对产品不需要原生 launcher3，可以使用自己的 launcher，且可以通过设置 android:directBootAware=“true” 属性来提前启动 launcher，从而绕过 directboot 机制导致的 launcher 启动时间晚的问题。

6.3.11 减少不必要的预装 apk

针对产品的需求来裁剪不必要的 apk，如果能裁剪掉原生大部分的 apk，能够节省 500ms。

6.3.12 AVB 优化

若预装应用太多，且预装到 System 分区，且这个时候打开了 AVB 功能，会严重影响 System 分区的负载，因为是边读边解密，针对这种情况需要将预装应用放到其他分区，如 OEM 分区或者 DATA 分区，尽量这部分不要添加 AVB 功能。防篡改重点防住 System 分区的内容即可，不是非常重要的数据没必要增加。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.3.13 并行加载驱动

当前许多驱动是默认编译成 ko，并放到 vendor_ram.disk 下，在启动的时候加载的，在以往，这些驱动是串行加载的，在 Android 13 谷歌支持了并行加载 vendor_ram.disk 里的驱动，修改方案如下。

在方案目录 mk 中增加如下配置。

```
BOARD_BOOTCONFIG += androidboot.load_modules_parallel=true
```

说明

该配置仅支持 Android 13 之后的版本，且默认已集成。

6.3.14 延迟加载非必须的模块和服务

在定制化开发后，对于自定义的服务，以及相关的外设，非必要情况下，不要放到 init:boot 阶段之前加载，尽量放到开机之后去完成。

例如，TP 模块在启动过程是不需要的，因此放到启动完成后加载。

```
on property:sys.boot_completed=1
   insmod /vendor/lib/modules/gslX680new.ko
```

对于自定义的需要在 system_server 中启动的服务，如果很耗时，可以放到线程池里面异步加载，参考如下。

```
SystemServerInitThreadPool.submit(() -> {
   TimingsTraceAndSlog traceLog = TimingsTraceAndSlog.newAsyncLog();
   traceLog.traceBegin("START_FONT_MANAGER_SERVICE");
   mSystemServiceManager.startService(new FontManagerService.Lifecycle(context, safeMode));
   traceLog.traceEnd();
}, "START_FONT_MANAGER_SERVICE");
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。